

```
[USER] ip
```

```
DESCRIPTION : Outil principal de gestion réseau (remplace ifconfig, route, arp)
POURQUOI : Commande moderne et unifiée pour toute la configuration réseau Linux.
           Gère les interfaces, les routes, les adresses, les tunnels, etc.
CONTEXTE : Configuration réseau, diagnostic, scripting réseau, administration.
SOUS-COMMANDES :
  ip addr          Gestion des adresses IP
  ip link          Gestion des interfaces réseau
  ip route         Gestion des routes
  ip neigh         Table ARP/NDP (voisins)
  ip rule          Règles de routage
  ip tunnel        Gestion des tunnels
  ip maddr         Adresses multicast
  ip netns         Namespaces réseau
OPTIONS GLOBALES :
  -4              IPv4 uniquement
  -6              IPv6 uniquement
  -s              Statistiques
  -h              Format lisible
  -j              Sortie JSON
  -c              Sortie colorée
  -o              Une ligne par objet
EXEMPLES :
$ ip addr show
$ ip addr add 192.168.1.10/24 dev eth0
$ ip link set eth0 up
$ ip route show
$ ip route add default via 192.168.1.1
$ sudo ip route add 10.0.0.0/8 via 192.168.1.254 dev eth0
$ ip neigh show
$ ip -s link show eth0
```

```
[USER] ifconfig
```

```
DESCRIPTION : Configure les interfaces réseau (commande historique, remplacée par ip)
POURQUOI : Encore présente sur de nombreux systèmes et documentation ancienne.
CONTEXTE : Systèmes anciens, compatibilité, scripts legacy.
OPTIONS :
  INTERFACE up/down      Active ou désactive une interface
  INTERFACE ADRESSE      Assigne une adresse IP
  netmask MASQUE         Définit le masque de sous-réseau
  -a                     Affiche toutes les interfaces (y compris inactives)
EXEMPLES :
$ ifconfig
$ ifconfig eth0
$ sudo ifconfig eth0 192.168.1.10 netmask 255.255.255.0
$ sudo ifconfig eth0 down
```

```
[USER] ss
```

```
DESCRIPTION : Affiche les sockets réseau (remplace netstat – plus rapide et détaillé)
POURQUOI : Voir toutes les connexions réseau, les ports en écoute, les sockets Unix.
CONTEXTE : Diagnostic réseau, audit de sécurité, monitoring de connexions.
OPTIONS :
  -t              Sockets TCP
  -u              Sockets UDP
  -l              Ports en écoute (LISTEN)
  -a              Toutes les connexions
  -n              Pas de résolution de nom
  -p              Affiche le processus (nécessite sudo pour tout voir)
  -e              Informations étendues
  -s              Statistiques résumées
  -4              IPv4 uniquement
  -6              IPv6 uniquement
  -x              Sockets Unix
  -r              Résolution des noms
```

```

FILTRES      :
sport :PORT      Port source
dport :PORT      Port destination
dst ADRESSE     Adresse destination
src ADRESSE     Adresse source
state established Connexions établies
state LISTEN     Ports en écoute
EXEMPLES      :
$ ss -tlnp
$ ss -an | grep :80
$ ss -s
$ sudo ss -tulnp
$ ss -t state established

```

```
[USER] netstat
```

```

DESCRIPTION  : Affiche les connexions réseau, les tables de routage, les statistiques
                (commande historique remplacée par ss et ip route)
POURQUOI    : Toujours présente sur les systèmes et dans la documentation existante.
CONTEXTE    : Diagnostic réseau, compatibilité, systèmes anciens.
OPTIONS      :
-t          Connexions TCP
-u          Connexions UDP
-l          Ports en écoute
-a          Toutes les connexions
-n          Pas de résolution de noms
-p          Affiche le PID et le programme
-r          Table de routage
-s          Statistiques par protocole
-i          Statistiques par interface
-e          Informations étendues
-c          Rafraîchissement continu
EXEMPLES     :
$ netstat -tlnp
$ netstat -an | grep ESTABLISHED
$ netstat -r
$ netstat -s | grep "segments"

```

```
[USER] ping
```

```

DESCRIPTION  : Teste la connectivité réseau vers un hôte en envoyant des paquets ICMP
POURQUOI    : Vérifier si un hôte est joignable, mesurer la latence, détecter la perte
                de paquets.
CONTEXTE    : Diagnostic réseau de base, mesure de latence, tests de connectivité.
OPTIONS      :
-c N        Envoie N paquets puis s'arrête
-i N        Intervalle entre paquets en secondes
-s N        Taille des paquets en octets
-t TTL      Time To Live des paquets
-W N        Timeout en secondes par réponse
-q          Mode silencieux (affiche seulement le résumé)
-f          Mode flood (très rapide, SUDO requis)
-4          Force IPv4
-6          Force IPv6
-I INTERFACE Interface source à utiliser
EXEMPLES     :
$ ping google.com
$ ping -c 4 192.168.1.1
$ ping -i 0.2 -c 100 serveur.local
$ sudo ping -f -c 1000 192.168.1.1

```

```
[USER] ping6
```

```

DESCRIPTION  : Comme ping mais pour IPv6
POURQUOI    : Tester la connectivité IPv6.
CONTEXTE    : Administration réseau IPv6, diagnostic.
EXEMPLES     :
$ ping6 ::1
$ ping6 -c 4 2001:db8::1

```

```
[USER] traceroute / tracepath
```

```

DESCRIPTION : Trace le chemin des paquets réseau vers une destination
POURQUOI : Identifier où les paquets sont bloqués ou ralentis sur le réseau.
CONTEXTE : Diagnostic de routing, identification de goulots réseau.
OPTIONS traceroute :
  -n Pas de résolution DNS (plus rapide)
  -m N TTL maximum (nombre de sauts)
  -w N Timeout en secondes
  -q N Nombre de paquets par saut
  -I Utilise ICMP (comme ping)
  -T Utilise TCP SYN (contourne certains firewalls)
  -p PORT Port destination
  -s ADRESSE Adresse source
OPTIONS tracepath :
  -n Pas de résolution DNS
  -b Affiche les deux (nom et IP)
  -l LEN Longueur initiale des paquets
  -p PORT Port de base UDP
EXEMPLES :
$ traceroute google.com
$ traceroute -n -T -p 80 serveur.com
$ tracepath 8.8.8.8

```

```
[USER] mtr
```

```

DESCRIPTION : Combinaison de ping et traceroute – monitoring continu du chemin réseau
POURQUOI : Voir en temps réel les latences et pertes de paquets à chaque saut.
            Bien plus informatif que traceroute seul.
CONTEXTE : Diagnostic réseau avancé, monitoring de qualité de lien.
OPTIONS :
  -n Pas de résolution DNS
  -r Mode rapport (non interactif)
  -c N Nombre de pings par saut
  -b Affiche les deux (nom et IP)
  -T Utilise TCP
  -P PORT Port TCP
  -u Utilise UDP
  --json Sortie JSON
EXEMPLES :
$ mtr google.com
$ mtr -rn -c 100 8.8.8.8
$ mtr --report --report-cycles 50 serveur.com

```

```
[USER] dig
```

```

DESCRIPTION : Requêtes DNS avancées – l'outil DNS le plus complet
POURQUOI : Interroger les serveurs DNS, déboguer la résolution, vérifier les
            enregistrements DNS (A, MX, NS, TXT, CNAME, etc.).
CONTEXTE : Administration DNS, débogage de résolution, vérification de propagation.
OPTIONS :
  @SERVEUR Interroge ce serveur DNS spécifique
  +short Affiche seulement la réponse courte
  +noall +answer Affiche seulement la section réponse
  +trace Trace la résolution depuis les root servers
  +nocmd +nocomments Sortie minimale
  -t TYPE Type d'enregistrement (A, MX, NS, TXT, CNAME, AAAA...)
  -x ADRESSE Recherche inverse (PTR record)
  +tcp Utilise TCP au lieu d'UDP
  -p PORT Port DNS (défaut: 53)
  +dnssec Demande les enregistrements DNSSEC
  +time=N Timeout en secondes
  +retry=N Nombre de tentatives
EXEMPLES :
$ dig google.com
$ dig @8.8.8.8 google.com A
$ dig google.com MX +short
$ dig -x 8.8.8.8
$ dig google.com NS +trace
$ dig TXT _dmarc.google.com

```

```
[USER] nslookup
```

DESCRIPTION : Requêtes DNS interactives ou non-interactives (plus ancien que dig)
 POURQUOI : Résolution DNS simple – présent sur presque tous les systèmes y compris Windows.
 CONTEXTE : Dépannage DNS cross-platform, vérification rapide.
 OPTIONS :
 -type=TYPE Type d'enregistrement (A, MX, NS, TXT...)
 -server=SERVEUR Serveur DNS à interroger
 EXEMPLES :
 \$ nslookup google.com
 \$ nslookup -type=MX google.com
 \$ nslookup 8.8.8.8

[USER] host

DESCRIPTION : Résolution DNS simple et directe
 POURQUOI : Commande DNS minimaliste – rapide pour les vérifications courantes.
 CONTEXTE : Vérification rapide de DNS, scripts.
 OPTIONS :
 -t TYPE Type d'enregistrement
 -a Tous les enregistrements
 -v Mode verbeux
 -4 Force IPv4
 -6 Force IPv6
 EXEMPLES :
 \$ host google.com
 \$ host -t MX google.com
 \$ host 8.8.8.8

[USER] curl

DESCRIPTION : Transfère des données depuis/vers des URLs (HTTP, HTTPS, FTP, SFTP...)
 POURQUOI : Outil HTTP en ligne de commande le plus puissant. Tester des APIs, télécharger des fichiers, envoyer des requêtes POST, déboguer HTTP.
 CONTEXTE : Tests d'API REST, téléchargements, scripts d'automatisation web.
 OPTIONS :
 -X MÉTHODE Méthode HTTP (GET, POST, PUT, DELETE, PATCH...)
 -H "En-tête: valeur" Ajoute un en-tête HTTP
 -d "données" Corps de requête (POST)
 -d @fichier Corps depuis un fichier
 -o FICHIER Sauvegarde la sortie dans un fichier
 -O Sauvegarde avec le nom original du fichier
 -L Suit les redirections
 -s Mode silencieux (pas de barre de progression)
 -S Affiche les erreurs même avec -s
 -v Mode verbeux (affiche les en-têtes)
 -I Affiche seulement les en-têtes de réponse (HEAD)
 -u USER:PASS Authentification HTTP Basic
 -b "cookies" Envoie des cookies
 -c FICHIER Sauvegarde les cookies dans un fichier
 -k Ignore les erreurs SSL
 --cert CERT Certificat client
 -x PROXY Utilise un proxy
 --retry N Nombre de tentatives
 --retry-delay N Délai entre tentatives
 -m N Timeout maximum en secondes
 -w FORMAT Format de sortie personnalisé
 --compressed Décompresse la réponse gzip/deflate
 -F "champ=@fichier" Upload multipart de fichier
 --json Content-Type: application/json (curl >=7.82)
 -Z Parallélisme (plusieurs URLs simultanées)
 EXEMPLES :
 \$ curl https://api.example.com/users
 \$ curl -X POST -H "Content-Type: application/json" -d '{"key":"val"}' https://api.example.com
 \$ curl -o fichier.zip https://example.com/fichier.zip
 \$ curl -u admin:password https://service.local/api
 \$ curl -v -k https://self-signed.example.com
 \$ curl -sS https://install.script.sh | bash
 \$ curl -w "%{http_code}" -o /dev/null -s https://example.com

[USER] wget

DESCRIPTION : Télécharge des fichiers depuis le web (HTTP, HTTPS, FTP)
 POURQUOI : Téléchargement de fichiers, mirroring de sites web. Supporte la

```

        reprise de téléchargements interrompus.
CONTEXTE   : Téléchargements, scripts, mirroring, récupération de sites.
OPTIONS    :
  -O FICHIER      Sauvegarde sous ce nom
  -c              Continue un téléchargement interrompu
  -r              Téléchargement récursif
  -l N            Profondeur de récursion
  -p              Télécharge toutes les ressources d'une page
  -k              Convertit les liens pour usage local
  -m              Mode miroir (-r -N -l inf --no-remove-listing)
  -q              Mode silencieux
  -v              Mode verbeux
  --user-agent="Agent"  Définit le user-agent
  --header="En-tête: val"  Ajoute un en-tête
  --no-check-certificate  Ignore les erreurs SSL
  --limit-rate=VITESSE    Limite la vitesse (ex: 500k)
  -N              Timestamp – ne télécharge que si plus récent
  -w N            Attend N secondes entre chaque requête
  -b              Continue en arrière-plan
  --accept=LISTE      Télécharge seulement ces types de fichiers
  --reject=LISTE      Refuse ces types de fichiers
EXEMPLES      :
$ wget https://example.com/fichier.tar.gz
$ wget -c https://exemple.com/gros_fichier.iso
$ wget -r -l 2 -p -k https://site.com
$ wget -q --show-progress https://example.com/fichier

```

```
[USER]  wget2
```

```

DESCRIPTION : Successeur de wget avec support HTTP/2, téléchargements parallèles
POURQUOI    : Plus rapide que wget grâce au parallélisme et HTTP/2.
CONTEXTE    : Téléchargements massifs, mirroring moderne.
EXEMPLES    :
$ wget2 --num-threads=8 https://example.com/fichier.iso

```

```
[USER]  ssh
```

```

DESCRIPTION : Client SSH – connexion sécurisée à distance à un serveur
POURQUOI    : Administration distante sécurisée. SSH est le protocole standard
              pour l'accès distant aux serveurs Linux.
CONTEXTE    : Administration de serveurs distants, tunnels, transferts sécurisés.
OPTIONS     :
  -p PORT      Port SSH du serveur (défaut: 22)
  -i CLEF      Fichier de clé privée
  -l USER      Nom d'utilisateur
  -X           Forwarding X11 (applications graphiques)
  -A           Forwarding de l'agent SSH
  -L PORT:HOTE:PORT_DISTANT  Tunnel local (Local Port Forwarding)
  -R PORT:HOTE:PORT_LOCAL    Tunnel distant (Remote Port Forwarding)
  -D PORT      Proxy SOCKS dynamique
  -N           Pas de commande distante (pour les tunnels)
  -f           Passe en arrière-plan
  -v / -vv / -vvv  Niveau de débogage (1, 2 ou 3)
  -o OPTION=VALEUR Options de configuration
  -J USER@JUMP_HOST  Jump host (bastion)
  -W HOTE:PORT  Tunnel transparent vers hôte:port
  -t           Force l'allocation d'un pseudo-terminal
  -q           Mode silencieux
  -C           Compression
  -4 / -6      Force IPv4 ou IPv6
EXEMPLES     :
$ ssh user@serveur.com
$ ssh -p 2222 -i ~/.ssh/id_rsa user@serveur.com
$ ssh -L 8080:localhost:80 user@serveur.com
$ ssh -D 1080 user@serveur.com
$ ssh -J user@bastion.com user@serveur_interne.com
$ ssh user@serveur.com 'df -h'

```

```
[USER]  scp
```

```

DESCRIPTION : Copie sécurisée de fichiers via SSH
POURQUOI    : Transférer des fichiers vers/depuis des serveurs distants de manière sécurisée.

```

```

CONTEXTE      : Déploiement de fichiers, récupération de logs, transfert de données.
OPTIONS       :
  -r           Copie récursive de répertoires
  -P PORT      Port SSH
  -i CLEF      Fichier de clé privée
  -p           Préserve les timestamps et permissions
  -C           Compression
  -v           Mode verbeux
  -q           Mode silencieux
  -l LIMITE    Limite de bande passante (Kbits/s)
  -3          Copie via le serveur local (entre deux distants)
EXEMPLES      :
$ scp fichier.txt user@serveur.com:/home/user/
$ scp -r /dossier/ user@serveur.com:/backup/
$ scp -P 2222 user@serveur.com:/var/log/syslog /tmp/
$ scp user@srv1:/fichier user@srv2:/destination

```

```
[USER] sftp
```

```

DESCRIPTION   : FTP sécurisé via SSH – interface interactive pour transfert de fichiers
POURQUOI      : Alternative à scp avec interface interactive de navigation.
CONTEXTE      : Transferts interactifs de fichiers, gestion de fichiers distants.
OPTIONS       :
  -P PORT      Port SSH
  -i CLEF      Fichier de clé privée
  -b FICHIER   Exécute des commandes depuis un fichier batch
COMMANDES SFTP :
  ls / lls     Liste le répertoire distant / local
  cd / lcd     Change le répertoire distant / local
  get FICHIER  Télécharge un fichier
  put FICHIER  Envoie un fichier
  mkdir / rmdir Crée / supprime un répertoire distant
  rm          Supprime un fichier distant
  bye / quit   Quitte
EXEMPLES      :
$ sftp user@serveur.com
$ sftp -P 2222 user@serveur.com

```

```
[USER] rsync
```

```

DESCRIPTION   : Synchronisation et transfert de fichiers locaux ou distants
POURQUOI      : Le meilleur outil de synchronisation – transfère seulement les
différences (delta), préserve les permissions, liens symboliques.
CONTEXTE      : Sauvegardes, déploiement, synchronisation de serveurs, mirroring.
OPTIONS       :
  -a           Mode archive (-rlptgoD) – préserve tout
  -v           Mode verbeux
  -z           Compression pendant le transfert
  -h           Format lisible pour les tailles
  -P          Affiche la progression + reprend les transferts
  -n / --dry-run Simulation – affiche ce qui serait fait
  --delete     Supprime les fichiers absents de la source
  --exclude=MOTIF Exclut des fichiers du transfert
  --include=MOTIF Inclut explicitement des fichiers
  --filter=RÈGLE Règles de filtrage avancées
  --backup     Fait une sauvegarde des fichiers modifiés
  --backup-dir=DIR Répertoire de sauvegarde
  --checksum   Compare par checksum (pas par taille/date)
  --bwlimit=KBPS Limite la bande passante
  -e "ssh -p 2222" Options SSH personnalisées
  --link-dest=DIR Hard links vers les fichiers inchangés (backups)
  -r          Récursif
  --progress   Barre de progression
  --stats      Affiche des statistiques finales
  -u          Passe si le fichier destination est plus récent
EXEMPLES      :
$ rsync -avzh /source/ /destination/
$ rsync -avz --delete /home/ user@serveur:/backup/home/
$ rsync -avzn --delete /src/ /dst/
$ rsync -avz --exclude="*.tmp" --exclude=".git/" /projet/ user@srv:/deploy/
$ rsync -avz --link-dest=/backup/dernier/ /data/ /backup/${date +%F}/

```

```
[USER] ssh-keygen
```

```
DESCRIPTION : Génère, gère et convertit des clés SSH
POURQUOI : Créer des paires de clés pour l'authentification sans mot de passe.
CONTEXTE : Configuration de l'accès SSH sécurisé, déploiement automatisé.
OPTIONS :
  -t TYPE          Type de clé (rsa, dsa, ecdsa, ed25519)
  -b BITS          Taille de clé (ex: 4096 pour RSA)
  -C "commentaire" Commentaire (souvent l'email)
  -f FICHIER       Fichier de sortie
  -N PASSPHRASE    Phrase de passe (vide = sans passphrase)
  -p              Change la passphrase
  -l              Affiche l'empreinte de la clé
  -F HOTE          Recherche un hôte dans known_hosts
  -R HOTE          Supprime un hôte de known_hosts
EXEMPLES :
$ ssh-keygen -t ed25519 -C "mon@email.com"
$ ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa_serveur
$ ssh-keygen -l -f ~/.ssh/id_ed25519.pub
```

```
[USER] ssh-copy-id
```

```
DESCRIPTION : Copie la clé publique SSH sur un serveur distant pour l'authentification
POURQUOI : Installer facilement une clé publique pour la connexion sans mot de passe.
CONTEXTE : Configuration de l'accès SSH par clé.
OPTIONS :
  -i CLEF          Fichier de clé publique à copier
  -p PORT          Port SSH
  -o OPTION        Options SSH supplémentaires
EXEMPLES :
$ ssh-copy-id user@serveur.com
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub user@serveur.com
$ ssh-copy-id -p 2222 user@serveur.com
```

```
[USER] nc / ncat / netcat
```

```
DESCRIPTION : Le "couteau suisse" du réseau – crée des connexions TCP/UDP arbitraires
POURQUOI : Tester des connexions, créer des serveurs simples, transférer des fichiers, déboguer des protocoles réseau.
CONTEXTE : Tests réseau, debug de protocoles, transfert de fichiers, CTF.
OPTIONS :
  -l              Mode écoute (serveur)
  -p PORT        Port à écouter ou source
  -u              Utilise UDP
  -v              Mode verbeux
  -n              Pas de résolution DNS
  -z              Mode scan (zero-I/O)
  -w N           Timeout en secondes
  -k              Reste en écoute après une connexion (ncat)
  -e CMD         Exécute une commande (DANGER en production!)
  --ssl           Utilise SSL/TLS (ncat)
EXEMPLES :
$ nc -zv serveur.com 80
$ nc -l -p 9999
$ nc serveur.com 9999 < fichier.txt
$ nc -zv serveur.com 1-1024 2>&1 | grep succeeded
```

```
[USER] nmap
```

```
DESCRIPTION : Scanner de réseau et d'audit de sécurité – le plus puissant du domaine
POURQUOI : Découvrir les hôtes actifs, les ports ouverts, les services, les OS.
CONTEXTE : Audit de sécurité, inventaire réseau, pentest, administration.
OPTIONS :
  -sS            Scan SYN (furtif) – SUDO requis
  -sT            Scan TCP connect – sans SUDO
  -sU            Scan UDP – SUDO requis
  -sV            Détection de version des services
  -O            Détection de l'OS – SUDO requis
  -A            Mode agressif (-sV -O --traceroute --script=default)
  -p PORTS       Ports à scanner (ex: 80,443 ou 1-1000 ou -)
  -p-           Tous les 65535 ports
```

-T0 à -T5	Vitesse (0=paranoïa, 3=normal, 5=insane)
-n	Pas de résolution DNS
--open	Affiche seulement les ports ouverts
--script=NOM	Exécute un script NSE
-oN FICHIER	Sortie normale vers fichier
-oX FICHIER	Sortie XML
-oG FICHIER	Sortie greppable
-Pn	Pas de ping préalable
-iL FICHIER	Cibles depuis un fichier
--exclude IP	Exclut ces IPs
-v / -vv	Verboosité

EXEMPLES :

```
$ nmap 192.168.1.0/24
$ sudo nmap -sS -sV -O -p- 192.168.1.1
$ nmap -sV --script=vuln 192.168.1.1
$ nmap -p 80,443,22 --open 192.168.1.0/24
$ sudo nmap -A -T4 serveur.com
```

[USER] tcpdump

DESCRIPTION : Capture et analyse le trafic réseau (analyseur de paquets en ligne de commande)
 POURQUOI : Voir exactement ce qui circule sur le réseau – débogage de protocoles, sécurité, analyse de trafic.

CONTEXTE : Diagnostic réseau avancé, sécurité, débogage d'applications réseau.

OPTIONS :

-i INTERFACE	Interface à capturer (ex: eth0, any)
-n	Pas de résolution de noms
-nn	Pas de résolution de noms ni de ports
-v / -vv / -vvv	Verboosité
-w FICHIER	Écrit les paquets dans un fichier pcap
-r FICHIER	Lit depuis un fichier pcap
-c N	Capture N paquets puis s'arrête
-s N	Taille de snapshot (0 = taille complète)
-A	Affiche le payload en ASCII
-x / -X	Affiche en hex / hex+ASCII
-e	Affiche les en-têtes Ethernet
-q	Mode concis
-l	Mode ligne (pour piping)
'FILTRE'	Filtre BPF (Berkeley Packet Filter)

FILTRES BPF COURANTS :

host IP	Trafic vers/depuis cette IP
port N	Trafic sur ce port
tcp port N	Trafic TCP sur ce port
src host IP	Depuis cette IP
dst host IP	Vers cette IP
not port 22	Exclut le port 22
net 192.168.1.0/24	Sous-réseau entier

EXEMPLES :

```
$ sudo tcpdump -i eth0 -nn
$ sudo tcpdump -i any port 80 -A
$ sudo tcpdump -w capture.pcap -i eth0
$ sudo tcpdump -r capture.pcap
$ sudo tcpdump -i eth0 'host 192.168.1.1 and tcp port 443'
$ sudo tcpdump -i eth0 'not port 22' -nn
```

[USER] arp / arping

DESCRIPTION : arp : affiche et modifie la table ARP
 arping : ping au niveau Ethernet/ARP
 POURQUOI : Diagnostiquer des problèmes de couche 2, détecter des conflits d'IP, vérifier si une IP est utilisée sur le réseau local.

CONTEXTE : Diagnostic réseau local, détection de conflits IP, sécurité.

OPTIONS arp :

-n	Affiche les adresses numériques
-a	Affiche toute la table ARP (format BSD)
-d IP	Supprime une entrée
-s IP MAC	Ajoute une entrée statique

OPTIONS arping :

-c N	Nombre de paquets
-I INTERFACE	Interface à utiliser
-D	Mode détection de doublon (duplicate address detection)
-f	Arrête au premier reply

EXEMPLES :


```
$ arp -n
$ arping -I eth0 192.168.1.1
$ arping -D -I eth0 192.168.1.100
```

```
[USER] ipcalc
```

```
DESCRIPTION : Calcule les informations d'un réseau IP (sous-réseau, broadcast, plage...)
POURQUOI : Calculer rapidement les paramètres d'un réseau à partir d'une adresse/masque.
CONTEXTE : Planification réseau, administration, scripts.
OPTIONS :
  -n Affiche l'adresse réseau
  -b Affiche l'adresse de broadcast
  -m Affiche le masque de sous-réseau
  -p Affiche le préfixe
  -s N1 N2... Divise en sous-réseaux de taille N
EXEMPLES :
$ ipcalc 192.168.1.0/24
$ ipcalc -s 50 30 20 192.168.1.0/24
```

```
[USER] nmcli
```

```
DESCRIPTION : Interface en ligne de commande pour NetworkManager
POURQUOI : Gérer les connexions réseau via NetworkManager – Wi-Fi, Ethernet, VPN, connexions mobiles.
CONTEXTE : Configuration réseau sur postes de travail et serveurs avec NM.
OPTIONS :
  device Gestion des interfaces
  connection Gestion des connexions
  general État général
  radio Gestion Wi-Fi/WWAN
  -t Mode concis
  -f CHAMPS Sélectionne des champs
  -p Mode "pretty"
EXEMPLES :
$ nmcli device status
$ nmcli connection show
$ nmcli device wifi list
$ sudo nmcli device wifi connect "MonWifi" password "MonMotDePasse"
$ nmcli general status
$ sudo nmcli connection add type ethernet ...
```

```
[USER] networkctl
```

```
DESCRIPTION : Interface pour systemd-networkd – gestion réseau systemd
POURQUOI : Gérer et interroger les interfaces réseau gérées par systemd-networkd.
CONTEXTE : Serveurs utilisant systemd-networkd au lieu de NetworkManager.
OPTIONS :
  list Liste les interfaces
  status INTERFACE État détaillé d'une interface
  lldp Informations LLDP (voisins réseau)
  up/down INTERFACE Active/désactive une interface
EXEMPLES :
$ networkctl list
$ networkctl status eth0
$ sudo networkctl up eth0
```

```
[USER] resolvectl
```

```
DESCRIPTION : Interroge et configure systemd-resolved (résolution DNS)
POURQUOI : Gérer la configuration DNS via systemd-resolved.
CONTEXTE : Systèmes utilisant systemd-resolved.
OPTIONS :
  status État de la résolution DNS
  query DOMAINE Résout un domaine
  dns Affiche les serveurs DNS configurés
  flush-caches Vide le cache DNS
EXEMPLES :
$ resolvectl status
$ resolvectl query google.com
$ sudo resolvectl flush-caches
```

```
[USER] nft
```

```
DESCRIPTION : Administration du pare-feu nftables (successeur d'iptables)
POURQUOI    : nftables est le nouveau framework de filtrage de paquets Linux –
              syntaxe plus claire, meilleures performances qu'iptables.
CONTEXTE    : Configuration de pare-feu, NAT, filtrage de paquets.
OPTIONS     :
  list ruleset      Affiche toutes les règles
  list tables       Affiche les tables
  add table         Crée une table
  add chain         Crée une chaîne
  add rule          Ajoute une règle
  delete rule       Supprime une règle
  flush ruleset     Vide toutes les règles
  -f FICHIER        Charge les règles depuis un fichier
EXEMPLES      :
$ sudo nft list ruleset
$ sudo nft add rule inet filter input tcp dport 80 accept
$ sudo nft -f /etc/nftables.conf
```

```
[SUDO] iptables / ip6tables
```

```
DESCRIPTION : Administration du pare-feu netfilter (iptables – historique)
POURQUOI    : Filtrage de paquets, NAT, routage. Encore très répandu malgré nftables.
CONTEXTE    : Pare-feu serveur, NAT/masquerade, redirection de ports.
OPTIONS     :
  -L          Liste les règles
  -L -n --line-numbers  Liste avec numéros de ligne et IPs
  -A CHAÎNE RÈGLE  Ajoute une règle en fin de chaîne
  -I CHAÎNE N RÈGLE  Insère une règle à la position N
  -D CHAÎNE N      Supprime la règle N
  -F CHAÎNE        Vide la chaîne
  -P CHAÎNE ACTION Définit la politique par défaut
  -t TABLE        Spécifie la table (filter, nat, mangle, raw)
  --dport PORT     Port destination
  --sport PORT     Port source
  -p PROTOCOLE     Protocole (tcp, udp, icmp)
  -s ADRESSE       Adresse source
  -d ADRESSE       Adresse destination
  -j ACTION        Action (ACCEPT, DROP, REJECT, LOG, MASQUERADE...)
  -i INTERFACE     Interface entrante
  -o INTERFACE     Interface sortante
EXEMPLES      :
$ sudo iptables -L -n --line-numbers
$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
$ sudo iptables -A INPUT -s 192.168.1.0/24 -j ACCEPT
$ sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
$ sudo iptables -A INPUT -p tcp --dport 22 -m limit --limit 3/min -j ACCEPT
```

```
[USER] firewall-cmd
```

```
DESCRIPTION : Interface pour firewalld – gestion de pare-feu par zones
POURQUOI    : Interface de haut niveau pour gérer le pare-feu sur Fedora/RHEL.
CONTEXTE    : Administration de pare-feu Fedora/RHEL/CentOS.
OPTIONS     :
  --state          État de firewalld
  --get-zones      Liste les zones
  --get-default-zone  Zone par défaut
  --list-all      Affiche toutes les règles
  --add-port=PORT/PROTO  Ouvre un port (temporaire)
  --add-port=PORT/PROTO --permanent  Ouvre un port (permanent)
  --add-service=SERVICE  Autorise un service
  --remove-port=PORT/PROTO  Ferme un port
  --reload         Recharge la configuration
  --zone=ZONE      Spécifie la zone
  --add-rich-rule="RÈGLE"  Règle avancée
EXEMPLES      :
$ sudo firewall-cmd --state
$ sudo firewall-cmd --list-all
$ sudo firewall-cmd --add-port=8080/tcp --permanent
$ sudo firewall-cmd --add-service=http --permanent
$ sudo firewall-cmd --reload
```

```
$ sudo firewall-cmd --zone=public --add-rich-rule='rule family=ipv4 source address=192.168.1.0/24 accept'
```

```
[USER] whois
```

```
DESCRIPTION : Interroge les informations d'enregistrement d'un domaine ou d'une IP
POURQUOI : Savoir à qui appartient un domaine ou une adresse IP.
CONTEXTE : Investigation, sécurité, administration DNS.
OPTIONS :
  -h SERVEUR : Serveur whois à interroger
  -p PORT : Port du serveur whois
EXEMPLES :
  $ whois google.com
  $ whois 8.8.8.8
  $ whois -h whois.ripe.net 192.168.1.1
```

```
[USER] nping
```

```
DESCRIPTION : Génération et analyse de paquets réseau avancée (fait partie de nmap)
POURQUOI : Tests réseau avancés avec contrôle total des paquets générés.
CONTEXTE : Tests de sécurité, débogage réseau avancé.
EXEMPLES :
  $ nping --tcp -p 80 serveur.com
  $ nping --udp -p 53 192.168.1.1
```

```
[USER] telnet
```

```
DESCRIPTION : Client Telnet – protocole non chiffré (obsolète pour connexions distantes)
POURQUOI : Encore utilisé pour tester la connectivité TCP à un port spécifique.
           Ne JAMAIS utiliser pour l'administration distante (non chiffré).
CONTEXTE : Test de connectivité TCP, débogage de services HTTP/SMTP/POP3.
EXEMPLES :
  $ telnet serveur.com 80
  $ telnet smtp.example.com 25
```

```
[USER] fribidi
```

```
DESCRIPTION : Implémentation de l'algorithme bidirectionnel Unicode
POURQUOI : Afficher correctement du texte arabe, hébreu et autres langues RTL.
CONTEXTE : Internationalisation, affichage de texte bidirectionnel.
EXEMPLES :
  $ echo "ابحرم" | fribidi
```

MODEMS, RÉSEAU MOBILE & OUTILS AVANCÉS

```
[USER] mmcli
```

```
DESCRIPTION : Interface en ligne de commande pour ModemManager – gestion des modems
               cellulaires 3G/4G/5G, cartes SIM, SMS, connexions mobiles.
POURQUOI : Gérer les modems USB/PCIe sur Linux : connexion mobile, SMS, infos SIM.
CONTEXTE : Laptops avec modem intégré, clés USB 4G/5G, IoT, serveurs avec failover mobile.
OPTIONS :
  -L : Liste tous les modems détectés
  -m N : Sélectionne le modem N (ex: -m 0)
  -m N --command=CMD : Envoie une commande AT au modem
  -m N --enable : Active le modem
  -m N --disable : Désactive le modem
  -m N --simple-connect="apn=APN" : Connecte avec cet APN
  -m N --simple-disconnect : Déconnecte
  -m N --messaging-list-sms : Liste les SMS
  -m N --messaging-create-sms="text=MSG,number="+336..." : Crée un SMS
  -s N --send : Envoie le SMS N
  -m N --location-get : Localisation GPS (si supporté)
  -m N --3gpp-scan : Scanne les réseaux disponibles
  --verbose : Mode verbeux
EXEMPLES :
  $ mmcli -L
  $ mmcli -m 0
```

```
$ mmcli -m 0 --simple-connect="apn=free,ip-type=ipv4v6"
$ mmcli -m 0 --simple-disconnect
$ mmcli -m 0 --messaging-list-sms
```

[SUDO] ModemManager

DESCRIPTION : Daemon de gestion des modems cellulaires (service systemd)
POURQUOI : Service de fond qui gère les modems 3G/4G/5G – doit tourner pour mmcli.
CONTEXTE : Systemd service, démarré automatiquement sur les systèmes avec modems.
EXEMPLES :
\$ sudo systemctl start ModemManager
\$ sudo systemctl enable ModemManager
\$ sudo systemctl status ModemManager

[USER] qmicli

DESCRIPTION : Interface CLI pour le protocole QMI (Qualcomm MSM Interface)
POURQUOI : Contrôle direct des modems Qualcomm via le protocole QMI natif.
Plus bas niveau que mmcli – accès aux fonctionnalités spécifiques Qualcomm.
CONTEXTE : Modems Qualcomm (très répandus dans les USB 4G/5G), IoT, embarqué.
OPTIONS :
-d DEVICE Périphérique QMI (ex: /dev/cdc-wdm0)
--dms-get-ids Obtenir les identifiants du modem (IMEI, IMSI...)
--nas-get-signal-strength Force du signal
--nas-get-home-network Réseau domicile
--nas-network-scan Scanne les réseaux disponibles
--wds-start-network="apn=APN" Démarrer une connexion données
--wds-stop-network Arrêter la connexion
--uim-get-card-status État de la carte SIM
EXEMPLES :
\$ sudo qmicli -d /dev/cdc-wdm0 --dms-get-ids
\$ sudo qmicli -d /dev/cdc-wdm0 --nas-get-signal-strength
\$ sudo qmicli -d /dev/cdc-wdm0 --uim-get-card-status

[USER] qmi-network

DESCRIPTION : Script d'aide pour établir/terminer des connexions réseau QMI
POURQUOI : Simplifier la gestion des connexions sur modems QMI.
CONTEXTE : Modems Qualcomm, connexion mobile automatisée.
EXEMPLES :
\$ sudo qmi-network /dev/cdc-wdm0 start
\$ sudo qmi-network /dev/cdc-wdm0 stop
\$ sudo qmi-network /dev/cdc-wdm0 status

[USER] qmi-firmware-update

DESCRIPTION : Met à jour le firmware d'un modem Qualcomm via QMI
POURQUOI : Mettre à jour le firmware des modems Qualcomm pour corriger des bugs
ou ajouter des bandes de fréquences.
CONTEXTE : Maintenance de modems Qualcomm, mise à jour firmware.
EXEMPLES :
\$ sudo qmi-firmware-update -d /dev/cdc-wdm0 --update firmware.bin

[USER] mbimcli

DESCRIPTION : Interface CLI pour le protocole MBIM (Mobile Broadband Interface Model)
POURQUOI : Contrôle des modems compatibles MBIM (standard USB 3.0+).
Alternative à QMI pour les modems supportant le standard MBIM.
CONTEXTE : Modems USB MBIM modernes, laptops avec modem intégré MBIM.
OPTIONS :
-d DEVICE Périphérique MBIM (ex: /dev/cdc-wdm0)
--query-device-caps Capacités du modem
--query-subscriber-ready-status État de la SIM
--query-signal-state Force du signal
--connect="apn=APN" Connexion avec APN
--disconnect Déconnexion
--query-ip-configuration Configuration IP obtenue
EXEMPLES :
\$ sudo mbimcli -d /dev/cdc-wdm0 --query-device-caps
\$ sudo mbimcli -d /dev/cdc-wdm0 --connect="apn=free"

```
$ sudo mbimcli -d /dev/cdc-wdm0 --query-signal-state
```

```
[USER] mbim-network
```

DESCRIPTION : Script d'aide pour gérer les connexions réseau MBIM
POURQUOI : Simplifier start/stop des connexions MBIM.
CONTEXTE : Modems MBIM, connexion mobile.
EXEMPLES :
\$ sudo mbim-network /dev/cdc-wdm0 start
\$ sudo mbim-network /dev/cdc-wdm0 stop

```
[USER] iw
```

DESCRIPTION : Configuration des interfaces Wi-Fi (remplace iwconfig)
POURQUOI : Gérer les interfaces Wi-Fi : scan, connexion, monitoring, statistiques.
CONTEXTE : Administration Wi-Fi, diagnostic, pentest réseau.
OPTIONS :
dev IFACE scan Scanne les réseaux Wi-Fi disponibles
dev IFACE link Informations de la connexion actuelle
dev IFACE connect SSID Connexion à un réseau ouvert
dev IFACE disconnect Déconnexion
dev IFACE set type monitor Mode moniteur (pour capture de paquets)
dev IFACE set type managed Mode normal (managed)
phy PHY info Informations sur le matériel Wi-Fi
dev IFACE station dump Liste les clients connectés (AP mode)
dev IFACE get station MAC Statistiques d'une station
reg get Affiche la réglementation radio
reg set CODE Définit le code pays pour la puissance radio
EXEMPLES :
\$ sudo iw dev wlan0 scan | grep SSID
\$ sudo iw dev wlan0 link
\$ sudo iw dev wlan0 set type monitor
\$ iw phy phy0 info

```
[USER] wpa_supplicant
```

DESCRIPTION : Daemon d'authentification WPA/WPA2/WPA3 pour les connexions Wi-Fi sécurisées
POURQUOI : Gère l'authentification Wi-Fi en dehors de NetworkManager – indispensable pour comprendre comment fonctionne le Wi-Fi sous Linux.
CONTEXTE : Connexion Wi-Fi manuelle, systèmes sans NetworkManager, IoT, embarqué.
OPTIONS :
-i INTERFACE Interface Wi-Fi
-c FICHIER Fichier de configuration
-D DRIVER Driver (nl80211, wext...)
-B Mode daemon (arrière-plan)
-d Mode débogage
-dd Mode débogage verbeux
-P FICHIER_PID Fichier PID
CONFIG EXEMPLE (/etc/wpa_supplicant/wpa_supplicant.conf) :
network={
 ssid="MonWifi"
 psk="MonMotDePasse"
 key_mgmt=WPA-PSK
}
EXEMPLES :
\$ sudo wpa_supplicant -B -i wlan0 -c /etc/wpa_supplicant.conf
\$ sudo wpa_supplicant -B -D nl80211 -i wlan0 -c /etc/wpa_supplicant/wpa_supplicant.conf

```
[USER] wpa_cli
```

DESCRIPTION : Interface CLI interactive pour contrôler wpa_supplicant
POURQUOI : Interagir avec wpa_supplicant en temps réel sans éditer les fichiers.
CONTEXTE : Debug Wi-Fi, gestion interactive des connexions.
COMMANDES INTERACTIVES :
scan Lance un scan Wi-Fi
scan_results Affiche les résultats du scan
status État de la connexion
add_network Ajoute un réseau
set_network N ssid "SSID" Configure le SSID
set_network N psk "PASSWORD" Configure le mot de passe
enable_network N Active le réseau N

select_network N	Sélectionne le réseau N
disconnect	Déconnecte
quit	Quitte

EXEMPLES :

```
$ sudo wpa_cli
$ sudo wpa_cli -i wlan0 status
$ sudo wpa_cli -i wlan0 scan_results
```

```
[USER] wpa_passphrase
```

DESCRIPTION : Génère un bloc de configuration wpa_supplicant avec PSK hashé
 POURQUOI : Ne pas stocker le mot de passe Wi-Fi en clair dans la config.
 CONTEXTE : Génération de configuration wpa_supplicant sécurisée.
 EXEMPLES :

```
$ wpa_passphrase "MonSSID" "MonMotDePasse"
$ wpa_passphrase "MonSSID" "MonMotDePasse" >> /etc/wpa_supplicant/wpa_supplicant.conf
```

```
[USER] ethtool
```

DESCRIPTION : Affiche et configure les paramètres des interfaces réseau Ethernet
 POURQUOI : Diagnostiquer les problèmes Ethernet : vitesse, duplex, autonegociation, Wake-on-LAN, statistiques d'erreurs, driver info.
 CONTEXTE : Diagnostic réseau, optimisation, Wake-on-LAN, administration.
 OPTIONS :

INTERFACE	Affiche les paramètres de l'interface
-i INTERFACE	Infos sur le driver
-S INTERFACE	Statistiques détaillées (erreurs, drops...)
-s INTERFACE speed N	Définit la vitesse (10/100/1000/10000)
-s INTERFACE duplex full/half	Définit le duplex
-s INTERFACE autoneg on/off	Active/désactive l'autonegociation
-s INTERFACE wol g	Active Wake-on-LAN (magic packet)
-s INTERFACE wol d	Désactive Wake-on-LAN
-t INTERFACE online	Test en ligne
-t INTERFACE offline	Test hors-ligne (COUPE la connexion!)
-k INTERFACE	Affiche les fonctionnalités (offload)
-K INTERFACE FEATURE on/off	Active/désactive une fonctionnalité
-g INTERFACE	Paramètres des ring buffers
-G INTERFACE rx N tx N	Définit les ring buffers
--show-eee INTERFACE	Affiche les paramètres EEE (Energy Efficient Ethernet)

EXEMPLES :

```
$ ethtool eth0
$ ethtool -i eth0
$ ethtool -S eth0 | grep -v ": 0"
$ sudo ethtool -s eth0 speed 1000 duplex full autoneg off
$ sudo ethtool -s eth0 wol g
```

```
[USER] mii-tool
```

DESCRIPTION : Affiche et configure les paramètres MII des interfaces réseau (ancien)
 POURQUOI : Vérifier l'état de la liaison physique Ethernet (link up/down, vitesse).
 CONTEXTE : Interfaces réseau anciennes, diagnostic de liaison physique.
 OPTIONS :

-v	Mode verbeux
-r	Reset de l'interface
-A	Annonce toutes les capacités

EXEMPLES :

```
$ mii-tool eth0
$ mii-tool -v eth0
```

```
[USER] mii-diag
```

DESCRIPTION : Diagnostic des interfaces réseau MII (encore plus ancien que mii-tool)
 POURQUOI : Diagnostic bas niveau de la couche physique réseau.
 CONTEXTE : Dépannage matériel réseau, interfaces anciennes.
 EXEMPLES :

```
$ mii-diag eth0
```

```
[USER] ifstat
```

DESCRIPTION : Affiche les statistiques réseau d'entrée/sortie en temps réel

POURQUOI : Surveiller le trafic par interface en temps réel.
CONTEXTE : Monitoring de bande passante, scripts de surveillance réseau.
OPTIONS :
 -i INTERFACES Filtre sur ces interfaces (séparées par ,)
 -a Toutes les interfaces
 -z Cache les interfaces sans trafic
 -n Pas de résolution de noms
 INTERVALLE Rafraîchissement en secondes
EXEMPLES :
 \$ ifstat 1
 \$ ifstat -i eth0,wlan0 2

[USER] nstat

DESCRIPTION : Statistiques réseau du noyau (comme netstat -s mais plus moderne)
POURQUOI : Accéder aux compteurs réseau du noyau : retransmissions TCP, erreurs...
CONTEXTE : Diagnostic réseau avancé, monitoring de qualité TCP.
OPTIONS :
 -a Affiche tous les compteurs (y compris à 0)
 -z Affiche seulement les compteurs non nuls
 -r Reset des compteurs
 -n Format numérique
EXEMPLES :
 \$ nstat
 \$ nstat -az | grep -i retrans
 \$ nstat -z

[USER] rtstat / ctstat / lnstat

DESCRIPTION : rtstat/ctstat : statistiques de routage et conntrack du noyau
 lnstat : statistiques réseau Linux génériques
POURQUOI : Monitorer les statistiques internes du noyau réseau.
CONTEXTE : Diagnostic avancé réseau, monitoring de performance kernel.
EXEMPLES :
 \$ lnstat -d
 \$ lnstat -f rt_cache -k entries,in_hit,in_miss
 \$ ctstat

[USER] iptstate

DESCRIPTION : Affiche les connexions suivies par netfilter/conntrack (comme top pour les connexions)
POURQUOI : Surveiller en temps réel les connexions trackées par le pare-feu.
CONTEXTE : Monitoring de pare-feu, diagnostic de connexions, sécurité.
OPTIONS :
 -t Trie par durée
 -b Trie par octets
 -S Affiche les totaux
 -f Filtre par protocole
EXEMPLES :
 \$ sudo iptstate
 \$ sudo iptstate -t

[USER] bridge

DESCRIPTION : Gestion des bridges réseau Linux (ponts réseau)
POURQUOI : Créer et gérer des bridges pour relier plusieurs interfaces réseau – indispensable pour la virtualisation et les conteneurs.
CONTEXTE : Virtualisation (KVM/QEMU), conteneurs, configuration réseau avancée.
OPTIONS :
 link show Affiche les interfaces membres des bridges
 fdb show Table de forwarding (MAC addresses)
 mdb show Table multicast
 vlan show VLANs configurés
 link add dev IFACE master BR Ajoute une interface à un bridge
 link del dev IFACE Retire une interface du bridge
EXEMPLES :
 \$ bridge link show
 \$ bridge fdb show dev br0
 \$ sudo bridge link add dev eth0 master br0

```
[USER] teamd / teaectl / teamnl
```

DESCRIPTION : teamd : daemon de teaming réseau (agrégation de liens)
teamdctl : contrôle du daemon teamd
teamnl : interface netlink pour teamd

POURQUOI : Agréger plusieurs interfaces réseau (bonding avancé) pour redondance et/ou augmentation de bande passante.

CONTEXTE : Serveurs avec plusieurs cartes réseau, haute disponibilité.

MODES TEAM :

roundrobin	Distribution cyclique des paquets
activebackup	Active/secours – bascule si l'active tombe
loadbalance	Équilibrage de charge avec hachage
broadcast	Envoie sur toutes les interfaces
lacp	IEEE 802.3ad LACP (agrégation standard)

EXEMPLES :

```
$ sudo teamd -r -d -c '{"runner":{"name":"activebackup"}}' -t team0
$ sudo teamdctl team0 state
$ sudo teamdctl team0 port add eth0
```

```
[USER] bond2team
```

DESCRIPTION : Convertit une configuration bonding en configuration teaming

POURQUOI : Migrer de l'ancien bonding Linux vers le nouveau teaming.

CONTEXTE : Migration de configuration réseau.

EXEMPLES :

```
$ bond2team --bonding_opts "mode=1 miimon=100"
```

```
[SUDO] pppd
```

DESCRIPTION : Daemon PPP (Point-to-Point Protocol) – connexions dial-up, DSL, VPN

POURQUOI : Établir des connexions PPP : modems, ADSL, tunnels PPPoE.

CONTEXTE : Modems dial-up, DSL, connexions série, VPN PPP.

OPTIONS :

DEVICE	Vitesse	Périphérique et vitesse (ex: /dev/ttyS0 115200)
user	NOM	Nom d'utilisateur
password	MDP	Mot de passe
noauth		Pas d'authentification
defaultroute		Ajoute une route par défaut
persist		Reconnecte automatiquement
debug		Mode débogage

EXEMPLES :

```
$ sudo pppd /dev/ttyUSB0 115200 user monuser noauth defaultroute persist
```

```
[SUDO] pppoe-discovery
```

DESCRIPTION : Découverte des serveurs PPPoE sur le réseau local

POURQUOI : Trouver les concentrateurs PPPoE (ADSL, fibre PPPoE) avant connexion.

CONTEXTE : Diagnostic PPPoE, configuration ADSL.

OPTIONS :

-I	INTERFACE	Interface à utiliser
-A		Envoie un PADI et attend les réponses

EXEMPLES :

```
$ sudo pppoe-discovery -I eth0
```

```
[SUDO] pptp
```

DESCRIPTION : Client VPN PPTP (Point-to-Point Tunneling Protocol)

POURQUOI : Se connecter à des serveurs VPN PPTP (Microsoft, anciens VPN d'entreprise).

CONTEXTE : VPN d'entreprise legacy, accès distant PPTP.

OPTIONS :

SERVEUR	Adresse du serveur PPTP	
--user	NOM	Nom d'utilisateur
--password	MDP	Mot de passe
--nolaunchpppd		Ne lance pas pppd automatiquement

EXEMPLES :

```
$ sudo pptp vpn.entreprise.com --user monuser
```

```
[USER] openvpn
```

DESCRIPTION : Client/serveur VPN OpenVPN – VPN SSL/TLS open source

POURQUOI : Connexion sécurisée à des VPN OpenVPN – très répandu en entreprise et services VPN commerciaux.

CONTEXTE : VPN d'entreprise, confidentialité, accès distant sécurisé.

OPTIONS :

- config FICHIER Fichier de configuration .ovpn
- daemon Lance en daemon
- log FICHIER Fichier de log
- status FICHIER Fichier de statut
- verb N Niveau de verbosité (0-9)
- auth-user-pass FICHIER Fichier user/password

EXEMPLES :

```
$ sudo openvpn --config mon_vpn.ovpn
$ sudo openvpn --config entreprise.ovpn --daemon --log /var/log/vpn.log
```

```
[USER] openconnect
```

DESCRIPTION : Client VPN pour Cisco AnyConnect, Juniper, Palo Alto, F5 GlobalAccess

POURQUOI : Remplaçant open source de Cisco AnyConnect – compatible avec les VPN d'entreprise Cisco sans nécessiter le client propriétaire.

CONTEXTE : VPN Cisco d'entreprise, accès distant sécurisé.

OPTIONS :

- u USER Nom d'utilisateur
- passwd-on-stdin Lit le mot de passe depuis stdin
- servercert SHA256:... Empreinte du certificat serveur
- b Mode arrière-plan
- protocol=PROTO Protocole (anyconnect, nc, gp, pulse, f5, array)
- interface NOM Nom de l'interface VPN
- s SCRIPT Script de configuration (vpnc-script)

EXEMPLES :

```
$ sudo openconnect vpn.entreprise.com -u monuser
$ echo "monmotdepasse" | sudo openconnect -u monuser --passwd-on-stdin vpn.com
```

```
[USER] vpnc / vpnc-disconnect
```

DESCRIPTION : Client VPN Cisco IPsec (groupe/pre-shared key)

POURQUOI : Se connecter aux VPN Cisco IPsec sans client propriétaire.

CONTEXTE : VPN Cisco d'entreprise utilisant IPsec avec group password.

OPTIONS :

- config FICHIER Fichier de configuration
- gateway SERVEUR Serveur VPN
- id GROUPE Identifiant de groupe
- username USER Nom d'utilisateur
- no-detach Ne passe pas en daemon

EXEMPLES :

```
$ sudo vpnc mon_vpn.conf
$ sudo vpnc-disconnect
```

```
[USER] ndctl
```

DESCRIPTION : Gestion des périphériques NVDIMM (mémoire persistante)

POURQUOI : Configurer et monitorer les modules de mémoire persistante (Intel Optane...).

CONTEXTE : Serveurs avec mémoire persistante NVDIMM, bases de données hautes performances.

OPTIONS :

- list Liste les périphériques NVDIMM
- create-namespace Crée un namespace
- disable-namespace Désactive un namespace
- enable-namespace Active un namespace
- destroy-namespace Supprime un namespace

EXEMPLES :

```
$ sudo ndctl list
$ sudo ndctl create-namespace -t pmem -m fsdax
```

```
[USER] ndptool
```

DESCRIPTION : Outil pour le protocole NDP (Neighbor Discovery Protocol) IPv6

POURQUOI : Tester et déboguer la découverte de voisins IPv6 (équivalent ARP pour IPv6).

CONTEXTE : Diagnostic réseau IPv6, test de routeurs, développement réseau.

OPTIONS :

- t TYPE Type de message NDP (rs, ra, ns, na, redirect)
- i INTERFACE Interface réseau

EXEMPLES :

```
$ sudo ndptool send -t rs -i eth0
$ sudo ndptool monitor -t ra -i eth0
```

```
[USER] genl-ctrl-list
```

DESCRIPTION : Liste les familles Generic Netlink disponibles dans le noyau
POURQUOI : Voir quels subsystemes netlink sont disponibles (Wi-Fi, nl80211, etc.).
CONTEXTE : Développement réseau, débogage netlink.
EXEMPLES :
\$ genl-ctrl-list

```
[USER] nl-* (suite de commandes libnl)
```

DESCRIPTION : Suite d'outils utilisant la bibliothèque libnl pour interagir avec
l'interface Netlink du noyau Linux.
POURQUOI : Accès direct aux fonctionnalités réseau du noyau via Netlink.
CONTEXTE : Administration réseau avancée, débogage, développement.

nl-addr-list / nl-addr-add / nl-addr-delete
→ Gestion des adresses IP via netlink
\$ nl-addr-list
\$ sudo nl-addr-add --dev eth0 local 192.168.1.10/24

nl-link-list / nl-link-set
→ Gestion des interfaces réseau
\$ nl-link-list
\$ sudo nl-link-set --dev eth0 --mtu 9000

nl-route-list / nl-route-add / nl-route-delete / nl-route-get
→ Gestion des routes
\$ nl-route-list
\$ sudo nl-route-add --dst 10.0.0.0/8 --gateway 192.168.1.254

nl-neigh-list / nl-neigh-add / nl-neigh-delete
→ Table ARP/NDP (voisins)
\$ nl-neigh-list

nl-qdisc-list / nl-class-list / nl-cls-list
→ Gestion QoS (Traffic Control)
\$ nl-qdisc-list

nl-monitor
→ Surveille les événements netlink en temps réel
\$ sudo nl-monitor

nl-rule-list
→ Règles de routage
\$ nl-rule-list

```
[USER] nf-* (suite conntrack/netfilter)
```

DESCRIPTION : Suite d'outils pour interagir avec le subsysteme conntrack de netfilter.
POURQUOI : Gérer la table de suivi des connexions du noyau (connection tracking).
CONTEXTE : Pare-feu avancé, NAT, diagnostic de connexions trackées.

nf-ct-list
→ Liste toutes les connexions trackées
\$ sudo nf-ct-list

nf-ct-add
→ Ajoute une entrée dans la table conntrack
\$ sudo nf-ct-add --src 192.168.1.1 --dst 8.8.8.8 --proto tcp ...

nf-ct-events
→ Surveille les événements conntrack en temps réel
\$ sudo nf-ct-events

nf-exp-list / nf-exp-add / nf-exp-delete
→ Gestion des "expectations" conntrack (FTP, SIP helpers)
\$ sudo nf-exp-list

nf-log

```
→ Affiche les logs netfilter
$ sudo nf-log
```

```
nf-monitor
→ Moniteur d'événements netfilter
$ sudo nf-monitor
```

```
nf-queue
→ Gestion des queues netfilter (NFQUEUE)
$ sudo nf-queue
```

```
[SUDO] slattach
```

```
DESCRIPTION : Attache une ligne série comme interface réseau (SLIP)
POURQUOI : Créer une connexion réseau sur une liaison série (SLIP, CSLIP).
CONTEXTE : Liaisons série, matériel embarqué, connexions legacy.
OPTIONS :
  -p PROTO Protocole (slip, cslip, adaptive...)
  -s VITESSE Vitesse (baud rate)
  -d Mode débogage
EXEMPLES :
$ sudo slattach -p cslip -s 115200 /dev/ttyS0
```

```
[SUDO] plipconfig
```

```
DESCRIPTION : Configure les paramètres d'une interface PLIP (Parallel Line IP)
POURQUOI : Réseau via port parallèle – complètement obsolète mais présent dans Linux.
CONTEXTE : Matériel très ancien, connexion entre deux PC via port parallèle.
EXEMPLES :
$ sudo plipconfig plip0 trigger 50
```

```
[USER] wsdd
```

```
DESCRIPTION : Web Services Discovery Daemon – rend Linux visible dans "Réseau" Windows
POURQUOI : Sans wsdd, les machines Linux avec Samba n'apparaissent pas dans l'explorateur Windows depuis Windows 10 (qui a supprimé NetBIOS).
CONTEXTE : Environnements mixtes Linux/Windows avec partages Samba.
OPTIONS :
  -i INTERFACE Interface à utiliser
  -4 / -6 IPv4 / IPv6 seulement
  -d Mode démon
  -s Mode silencieux
EXEMPLES :
$ sudo wsdd -i eth0
$ sudo systemctl enable wsdd && sudo systemctl start wsdd
```

```
[USER] tcpslice
```

```
DESCRIPTION : Extrait des portions d'un fichier pcap selon des critères de temps
POURQUOI : Découper des captures réseau volumineuses en extraits temporels.
CONTEXTE : Analyse forensics, traitement de captures réseau.
OPTIONS :
  DEBUT FIN Timestamps de début et de fin
  -r Affiche les temps relatifs
EXEMPLES :
$ tcpslice 12:00:00 13:00:00 capture.pcap > heure_midi.pcap
$ tcpslice -r 0 3600 capture.pcap > premiere_heure.pcap
```